

Two Sum

LeetCode Q1

Given:- an array with int values,
[2, 7, 11, 15]
a target value eg:- 9

Return array with index numbers of values
whose sum gets you the target value

In this case, $2 + 7 = 9 \rightarrow$ target

So output array = [0, 1]

index(2) $\hookrightarrow$ index(7)

Coding Approach

(1) Brute Force,

arr = [1, 2, 3, 4, 5] ; target = 7

approach

```
for(int i=0; i<arr.size; i++){  
    for(int j=0; j<arr.size; j++){  
        if (nums[i]+nums[j]==target)  
            return {i,j};  
    }  
}
```

Issues with this approach:-

Time Complexity :- $O(n^2)$

Can we improve this definitely,

(2) The hashmap approach

what if instead of storing index via iteration, we store it using some function. Vectors don't have such function. Thus, we choose HashMap.

Reason:-
- hashmap has Key, value pairs.
- search complexity = $O(1)$ avg. case
worst case = $O(n)$
- has find function to search element.

Approach:-

(i) hashmap declaration

`unordered_map<int, int> numMap;`

(ii) iteration (single for loop)

(iii) set int $i=0$ $\nearrow$; int $j = \text{target} - \text{nums}[i]$

(iv) if statement

if true, find index and append to hashmap

else, return empty map

Diagram:-

[2, 7, 11, 15] target = 9
 $\downarrow$ $\downarrow$ $\downarrow$
nums[i] $j = \text{target} - \text{nums}[i]$ $2 + 7 = 9$

if j exists already in hashmap

return {numMap[j], i}

$\hookrightarrow$ index of j

else,

numMap[nums[i]] = i;

Finally return array/vector, empty

if nothing exists or satisfies condition.

Final Code:-

```
class Solution {  
public:  
    vector<int> twoSum(vector<int>& nums, int target) {  
        // map to store : <value, index>  
        unordered_map<int, int> numMap;  
  
        for(int i=0; i<nums.size(); i++){  
            int j = target - nums[i];      // setting j as complement  
  
            // checking if j exists  
            if(numMap.find(j) != numMap.end()){  
                return {numMap[j], i};  
            }  
            // otherwise add current number and its index to map  
            numMap[nums[i]] = i;  
        }  
        // return empty vector if no solution found  
        return {};  
    }  
};
```